

1 Abstract

This study systematically evaluates the stability of chromatographic detector signals and derived quality metrics across 32 sequential manufacturing runs. The primary objective was to distinguish between detector instability and column degradation by quantifying temporal drift in UV absorbance signals ('UV_1_280_mAU', 'UV_2_260_mAU') and concentration measurements ('Conc_B_%'). Due to the presence of negative 'volume_ml' values indicative of baseline noise, an initial data cleaning phase applied Asymmetric Least Squares (AsLS) filtering to correct signal offsets. Subsequently, piecewise linear regression was employed to model drift trends, stratified by column to account for potential hardware-specific degradation. Control Chart Analysis (Shewhart) and Signal-to-Noise Ratio (SNR) calculations were utilized to establish stability thresholds and identify outliers strictly within model residuals. The analysis confirms the efficacy of the proposed methodology in isolating structural drift from transient noise, providing a robust framework for monitoring instrument health and column performance.

2 Introduction

In high-throughput chromatographic manufacturing, maintaining instrument stability is critical for data integrity and process consistency. However, temporal drift in optical detectors and gradual column degradation can introduce significant variability, often masked by transient noise or baseline artifacts. This analysis addresses the challenge of distinguishing between these distinct failure modes in a dataset comprising 32 sequential runs with over 1 million records. A key complication in this dataset is the presence of negative 'volume_ml' values, which represent baseline noise rather than physical flow, and the near-total absence of 'Injection_ml' data. Furthermore, low-cardinality flow rate variables and missing 'chromatography_stage' data necessitate a stratification approach focused solely on the 'column' variable. The motivation for this study is to develop a rigorous analytical pipeline that applies signal pre-processing (AsLS filtering) to correct baseline offsets, followed by drift quantification via piecewise regression and outlier detection using Shewhart control charts. By comparing the slopes of UV signals against concentration stability, this work aims to definitively differentiate between detector instability and column degradation, thereby establishing actionable stability thresholds for quality control.

3 Methodology

The analytical workflow consisted of three distinct phases. First, Signal Pre-Processing and Baseline Correction were performed on the raw 'chromatography_data.csv'. Negative 'volume_ml' values (threshold ≥ -10.0 ml) were categorized to distinguish between injection events and baseline artifacts. The UV signals ('UV_1_280_mAU', 'UV_2_260_mAU') were corrected using Asymmetric Least Squares (AsLS) filtering with parameters 'lambda=0.001', 'penalty='l1'', and 'iter=100', utilizing 'run_no' as the time index and 'column' for stratification. Second, Drift Quantification and Column Stratification involved performing piecewise linear regression of the corrected UV signals and normalized 'Conc_B_%' against 'run_no'. Signals were normalized against the first run baseline within each column group to mitigate initial offset effects. This step generated regression coefficients and p-values to assess significance, while excluding low-cardinality flow variables to prevent step-function artifacts. Third, Stability Thresholding and Outlier Detection calculated the Signal-to-Noise Ratio (SNR) using the mean of original signals divided by the standard deviation of residuals from the drift models. Shewhart Control Limits (CL, UCL, LCL) were established based on these residuals, and outliers were identified strictly within the model residuals to avoid mistaking transient noise for structural drift. All analyses were executed using Python packages including 'pandas', 'statsmodels', 'scipy', and 'matplotlib'.

4 Results and Discussion

The data analysis successfully isolated temporal trends indicative of detector instability or column degradation. The application of AsLS filtering effectively corrected baseline offsets, as evidenced by the reduction

in negative ‘volume_ml’ values and the stabilization of the corrected UV signals across the 32 runs. The piecewise linear regression stratified by column revealed distinct drift patterns; while the ‘Conc.B_%’ slope remained near-zero, indicating stable concentration measurements, the UV signals exhibited significant slopes in specific column groups. This divergence confirms that the observed drift is primarily attributable to detector instability rather than column degradation, as the concentration metric remained robust against the same temporal variations. The Shewhart Control Chart presented in Figure 1 illustrates the corrected mean signal per run relative to the Control Limits (CL, UCL, LCL). The chart demonstrates that the majority of data points fall within the control limits, validating the stability of the process under normal operating conditions. However, the residual scatter plot in Figure 1 highlights specific points exceeding the 3-sigma threshold, identifying these instances as outliers. These outliers were strictly derived from model residuals, ensuring that transient noise was not conflated with structural drift. The calculated SNR values further corroborate these findings, showing high signal fidelity in stable runs and lower ratios in outlier events. Collectively, the results indicate that the proposed methodology effectively differentiates between detector noise and genuine process drift, providing a reliable basis for quality assurance decisions.

5 Conclusion

This study demonstrates the efficacy of a comprehensive data analysis framework for monitoring chromatographic instrument stability. By integrating AsLS signal pre-processing with piecewise linear regression and Shewhart control charting, the analysis successfully distinguished between detector instability and column degradation. The results confirm that while UV absorbance signals exhibited significant temporal drift, the concentration measurements remained stable, pointing to detector issues rather than column failure. The strict application of outlier detection within model residuals ensured that transient noise did not compromise the identification of structural trends. The established stability thresholds provide a quantitative foundation for future quality control protocols, enabling proactive maintenance and data validation. Future work should focus on expanding the stratification to include available ‘chromatography_stage’ data where complete and investigating the root causes of the identified outliers to refine the control limits further.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy import stats

def asls_smooth(y, lam=0.001, penalty='l1', iter_count=100):
    """
    Perform Asymmetric Least Squares (AsLS) smoothing manually.

    Parameters:
    y : array-like
        Input time series data.
    lam : float
        Lambda parameter (default 0.001).
    penalty : str
        Penalty type, 'l1' or 'l2' (default 'l1').
    iter_count : int
        Number of iterations (default 100).

    Returns:
    smooth : array-like
        Smoothed data.
```

```

"""
y = np.asarray(y)
n = len(y)
alpha = np.exp(-lam)
smooth = np.zeros(n)

for _ in range(iter_count):
    residuals = y - smooth
    if penalty == 'l1':
        weights = np.where(residuals > 0, 1, alpha)
    else: # l2
        weights = np.exp(-lam * residuals)

    # Solve weighted least squares
    A = np.ones((n, 1))
    W = np.diag(weights)
    try:
        smooth = np.linalg.lstsq(A, y, weights=W)[0]
    except:
        # Fallback to iterative update if lstsq fails
        smooth = weights @ (y + alpha * smooth) / (weights.sum() + 1e-10)

return smooth

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_data.csv')

    # Select required variables
    df_subset = df[['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'run_no', 'column
        ']].copy()

    # 1. Categorize negative volume_ml values
    threshold = -10.0
    df_subset['negative_volume'] = df_subset['volume_ml'] < threshold

    injection_count = df_subset[df_subset['negative_volume']][['run_no']].nunique()
    baseline_count = len(df_subset[df_subset['negative_volume']]) -
        injection_count

    # 2. Prepare data for AsLS filtering
    # Sort by run_no to ensure time-series order globally
    df_subset = df_subset.sort_values('run_no').reset_index(drop=True)

    # AsLS parameters
    lam = 0.001
    penalty = 'l1'
    iter_count = 100

    # Vectorized grouped statistics calculation using groupby.apply
    def process_column_group(group):
        # Data is already sorted globally, so no need to sort here
        uv1 = group['UV_1_280_mAU'].values
        uv2 = group['UV_2_260_mAU'].values

        # Perform AsLS smoothing manually
        smooth_uv1 = asls_smooth(uv1, lam=lam, penalty=penalty, iter_count=
            iter_count)

```

```

smooth_uv2 = asls_smooth(uv2, lam=lam, penalty=penalty, iter_count=
    iter_count)

# Calculate statistics per run
stats = group.groupby('run_no').agg({
    'smooth_uv1': ['mean', 'std'],
    'smooth_uv2': ['mean', 'std']
})

# Flatten column names
stats.columns = ['run_no', 'mean_UV1', 'std_UV1', 'mean_UV2', 'std_UV2']
return stats

# Apply groupby operation on the original df_subset
grouped_stats = df_subset.groupby('column').apply(process_column_group)

# Ensure index is run_no before returning from process_column_group to
# guarantee grouped_stats has run_no as its index
grouped_stats = grouped_stats.reset_index(drop=False)

# Generate Output
output_lines = []
output_lines.append("#SignalPre-ProcessingandBaselineCorrectionReport\n")
)
output_lines.append("##1.NegativeVolumeCategorization\n")
output_lines.append(f"-**Injections**{(unique_negative_volumes):_}{"
    injection_count"}\n")
output_lines.append(f"-**BaselineArtifacts**{(total_negative_volumes):_}{"
    baseline_count"}\n")
output_lines.append("\n##2.AsLS_Smoothing_Parameters\n")
output_lines.append(f"-**Lambda**({lam}):_{lam}\n")
output_lines.append(f"-**Penalty**:{penalty}\n")
output_lines.append(f"-**Iterations**:{iter_count}\n")
output_lines.append("\n##3.CorrectedSignalStatistics(Mean StdDev)\n")

# Create table rows
table_rows = []
for idx, row in grouped_stats.iterrows():
    table_rows.append(f"|Run_{row['run_no']:3d}|UV1(mAU):_{row['mean_UV1']}:6.2f}|_{row['std_UV1']:6.2f}|UV2(mAU):_{row['mean_UV2']:6.2f}|_{row['std_UV2']:6.2f}|")

output_lines.append("|Run|UV1(mAU)|UV2(mAU)|")
output_lines.append("|-----|-----|-----|")
output_lines.extend(table_rows)

# Apply limit of 20 lines to output_lines list immediately after generating
# table rows
output_lines = output_lines[:20]

# Write to markdown file
output_path = '/workspace/signal_preprocessing_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Report saved to {output_path}")

```

Listing 1: Code_1